# Multiplicación de matrices de forma secuencial y paralela

Caso de Estudio 1 — Computación de Alto Rendimiento

Daniel Toro Soto , Juan Camilo Galvis

Marzo 2026

**Resumen**

El presente documento describe el proceso de implementación y análisis comparativo del algoritmo de multiplicación de matrices cuadradas en el lenguaje C, evaluado bajo cuatro estrategias de ejecución: secuencial ingenuo, secuencial con optimizaciones del compilador (`-O2`), secuencial con optimización de caché (transposición de matrices) y paralela mediante hilos POSIX (`pthread`) y procesos (`fork`). Las pruebas se realizaron sobre matrices cuadradas de dimensiones entre 500 y 4000, con 10 repeticiones por configuración, variando el número de trabajadores (2, 4, 8 y 16) para las versiones paralelas. Los resultados se obtuvieron sobre un Apple Mac Mini con procesador M4 de 10 núcleos y 16 GB de memoria unificada. Se analizan tiempos de ejecución, speedup y el impacto de la localidad de caché, obteniendo conclusiones sobre la eficiencia real del paralelismo en hardware Apple Silicon.

# Índice

**1. Introducción** **3**

**2. Marco Conceptual** **3**
2.1. High Performance Computing (HPC) . . . 3
2.2. Multiplicación matricial . . . 3
2.3. Complejidad Computacional y localidad de caché . . . 3
2.4. Programación paralela . . . 3
2.5. Hilos vs. Procesos . . . 4
2.6. Speedup . . . 4

**3. Marco Contextual** **4**
3.1. Características de la máquina . . . 4
3.2. Desarrollo . . . 4
3.3. Sistema de compilación (Makefile) . . . 5
3.4. Sistema de benchmarking con checkpoints . . . 5
3.5. Problema de medición: CPU acumulado vs. tiempo de pared . . . 6

**4. Pruebas** **6**
4.1. Secuencial . . . 6
4.2. Secuencial con flag . . . 7
4.3. Secuencial con flag . . . 7
4.4. Secuencial con flag . . . 7
4.5. Secuencial con flag . . . 7
4.6. Secuencial con flag . . . 8
4.7. Secuencial con flag . . . 8
4.8. Secuencial con flag . . . 8
4.9. Secuencial con flag . . . 8
4.10. Secuencial con flag . . . 9
4.11. Secuencial con flag . . . 9

**5. Gráficas comparativas** **9**

**6. Análisis de resultados** **12**
6.1. Impacto del compilador (`-O2`) . . . . . 12
6.2. Optimización de caché (transpuesta) . . . . . 12
6.3. Paralelismo con hilos . . . . . 12
6.4. Paralelismo con procesos . . . . . 13

**7. Conclusiones** **13**

**8. Bibliografía** **14**

# 1. Introducción

A lo largo de los años la computación ha evolucionado hasta el punto de poder resolver problemas de grandes dimensiones en tiempos cada vez más reducidos. La multiplicación de matrices es uno de los problemas de referencia en esta área: a pesar de su sencillez conceptual, su complejidad $\mathcal{O}(n^3)$ lo convierte en un candidato ideal para evaluar estrategias de paralelización y optimización.

Como señalan Qaz Wiki (2020), "se ha invertido mucho trabajo en hacer que los algoritmos de multiplicación de matrices sean eficientes [...] Se han diseñado muchos algoritmos diferentes para multiplicar matrices en diferentes tipos de hardware, incluidos sistemas paralelos y distribuidos, donde el trabajo computacional se distribuye en múltiples procesadores". En este estudio se implementaron cinco variantes del algoritmo en C: una versión base secuencial, la misma versión compilada con la bandera de optimización `-O2`, una versión que explota la localidad de caché transponiendo la segunda matriz, y versiones paralelas con hilos y procesos.

Con base en el concepto de *speedup* —la razón entre el tiempo secuencial y el tiempo paralelo— se busca cuantificar la ganancia real obtenida al paralelizar, considerando el efecto de la arquitectura Apple M4 (núcleos heterogéneos de rendimiento y eficiencia) y el papel del subsistema de memoria unificada de alta velocidad de esta plataforma.

Finalmente, el estudio busca determinar a partir de qué dimensión de matriz el paralelismo compensa su overhead y cuántos trabajadores resultan óptimos en esta arquitectura particular.

# 2. Marco Conceptual

## 2.1. High Performance Computing (HPC)

Cuando hablamos de HPC, "hacemos referencia a un campo de la computación actual que da solución a problemas tecnológicos muy complejos y que involucran un gran volumen de cálculos o de coste computacional" (López, 2017). En este caso de estudio, el aumento de la dimensión de las matrices implica un crecimiento cúbico del trabajo computacional, lo que justifica el uso de técnicas de HPC como la paralelización y la optimización de accesos a memoria.

## 2.2. Multiplicación matricial

Dadas dos matrices cuadradas $A$ y $B$ de dimensión $n$:

$$A := (a_{ij})_{n\times n} \qquad B := (b_{ij})_{n\times n}$$

El producto $C = AB$ genera una matriz resultado $C := (c_{ij})_{n\times n}$ donde:

$$c_{ij} = \sum_{k=1}^{n} a_{ik} \cdot b_{kj}$$

Esta definición expone directamente los tres bucles anidados del algoritmo ingenuo, con complejidad temporal $\mathcal{O}(n^3)$ y complejidad espacial $\mathcal{O}(n^2)$.

## 2.3. Complejidad Computacional y localidad de caché

La Teoría de la Complejidad estudia la eficiencia de los algoritmos en función de los recursos que utiliza (Cortéz, 2004). Para matrices de gran dimensión, el patrón de acceso a memoria es crítico: el algoritmo ingenuo accede a la columna $j$ de $B$ iterando sobre $k$, lo que produce fallos de caché continuos al no respetar la disposición en fila mayor (*row-major*) del lenguaje C. Transponer $B$ antes de multiplicar convierte esos accesos en secuenciales, mejorando significativamente el rendimiento para matrices grandes.

## 2.4. Programación paralela

"La computación paralela es el uso de múltiples recursos computacionales para resolver un problema. Se distingue de la computación secuencial en que varias operaciones pueden ocurrir simultáneamente" (Ferestrepoca, 2019). En este estudio se emplean dos mecanismos de paralelismo:

- **Hilos POSIX (`pthread`)**: comparten el espacio de memoria del proceso, con bajo overhead de creación y comunicación.

- **Procesos (`fork`)**: cada proceso tiene su propio espacio de memoria; la matriz resultado se comparte mediante `mmap(MAP_SHARED | MAP_ANONYMOUS)`.

### 2.5. Hilos vs. Procesos

"La diferencia fundamental entre hilos y procesos es que un proceso dispone de un espacio de memoria separado, mientras que un hilo no. De esta forma los hilos pueden trabajar directamente con las variables creadas por el proceso padre, mientras que los procesos hijos no pueden hacerlo" (Montero & Antunez, 2011). Esta diferencia se refleja en el overhead de cada enfoque y, en consecuencia, en su speedup efectivo.

### 2.6. Speedup

El speedup representa la ganancia en rendimiento de la versión paralela respecto a la secuencial:

$$S_p = \frac{T^*(n)}{T_p(n)}$$

donde $T^*(n)$ es el tiempo de ejecución secuencial y $T_p(n)$ el tiempo con $p$ trabajadores. El speedup ideal es lineal ($S_p = p$), pero en la práctica se ve limitado por porciones secuenciales (Ley de Amdahl) y por el overhead de sincronización.

## 3. Marco Contextual

### 3.1. Características de la máquina

Las pruebas se realizaron sobre el siguiente hardware:

- **Equipo:** Apple Mac Mini (2024)
- **Procesador:** Apple M4 — 10 núcleos (4 de rendimiento + 6 de eficiencia)
- **GPU integrada:** 10 núcleos
- **Memoria:** 16 GB RAM unificada LPDDR5X (banda ancha ≈120 GB/s)
- **Almacenamiento:** 512 GB SSD NVMe
- **Sistema operativo:** macOS Sequoia 15.x (Darwin 25.x)
- **Compilador:** Apple Clang 16 (LLVM)

La arquitectura Apple M4 presenta características relevantes para este estudio: la memoria unificada elimina la transferencia entre CPU y memoria separada, y los núcleos heterogéneos (P-cores y E-cores) implican que el speedup al pasar de 8 a 16 trabajadores puede verse afectado por la diferencia de rendimiento entre tipos de núcleo.

Las pruebas evaluaron matrices cuadradas de dimensiones: **500, 1000, 1300, 1600, 2000, 2300, 2600, 3000, 3300, 3600 y 4000**, con **10 repeticiones** por configuración para obtener promedios estadísticamente representativos.

### 3.2. Desarrollo

Las cinco implementaciones en C comparten la misma estructura base: generación aleatoria de matrices con enteros entre 0 y 99 (librería `stdlib.h`), multiplicación y medición del tiempo de CPU mediante `getrusage(RUSAGE_SELF, ...)` con `ru_utime`. El tiempo impreso en stdout corresponde al tiempo de usuario en segundos con 6 decimales.

`secuential` Algoritmo ingenuo $\mathcal{O}(n^3)$ con tres bucles anidados. Accede a $B[k][j]$, generando saltos de caché en la dimensión $k$.

`secuentialFlags` Mismo código fuente que `secuential`, compilado con `-O2` para medir el impacto de las optimizaciones automáticas del compilador (vectorización, reordenamiento de instrucciones, etc.).

`memory` Transpone la matriz $B$ antes de multiplicar, de modo que el acceso interno es $B^T[j][k]$, convirtiendo todos los accesos en secuenciales en memoria.

`threads` Divide las filas de $C$ equitativamente entre $p$ hilos POSIX. Cada hilo calcula su bloque de filas de manera independiente sobre matrices compartidas.

`multiprocessing` Crea $p$ procesos hijos con `fork()`. La matriz resultado $C$ se ubica en memoria compartida anónima (`mmap`); las matrices $A$ y $B$ se heredan por *copy-on-write*. El tiempo se mide con `RUSAGE_SELF + RUSAGE_CHILDREN`.

El código fuente del proyecto se encuentra en: https://github.com/pausa11/HPC

## 3.3. Sistema de compilación (Makefile)

El proyecto utiliza un `Makefile` que compila los cinco binarios en el directorio `output/`:

```
make              # Compila los 5 binarios
make clean        # Elimina output/
```

Un aspecto clave del diseño es la estrategia deliberada de flags:

- `secuential` y `memory` compilan **sin ninguna bandera**, con el compilador en modo por defecto. Esto garantiza que los resultados reflejan el algoritmo puro, sin interferencia del optimizador.
- `secuentialFlags` compila el **mismo código fuente** que `secuential` pero con `-O2`, aislando el efecto de las optimizaciones automáticas del compilador.
- `threads` agrega únicamente `-pthread`, la bandera requerida por la librería POSIX de hilos.
- `multiprocessing` compila sin flags adicionales, ya que `fork()` y `mmap()` son llamadas al sistema estándar de POSIX.

Esta separación permite comparar el impacto de cada factor de forma independiente: algoritmo base, optimización de compilador, localidad de caché e hilos/procesos.

## 3.4. Sistema de benchmarking con checkpoints

Dado que las corridas completas pueden tardar varias horas (matrices de hasta 4000×4000 con 10 repeticiones), el script `scripts/RunAll.sh` implementa un sistema de **reanudación automática** basado en checkpoints.

**Flujo de ejecución:**

1. Al iniciar, el script busca en `stats/<hostname>/` si existe un directorio de corrida sin el sufijo `.done`. Si existe, lo reanuda; si no, crea uno nuevo con timestamp (`YYYYMMDD_HHMMSS`).
2. Cada medición exitosa registra una clave en `checkpoint.log` con el formato `<tipo>,<tamaño>,<config>,<run>` (ej. `threads,2000,n8,run3`).
3. La función `run_safe()` verifica antes de cada ejecución si esa clave ya existe en el log. De ser así, la omite (`[SKIP]`). Si el programa falla con código de error distinto de cero (por ejemplo, un `SIGSEGV` por falta de memoria en matrices muy grandes), registra el aviso pero continúa con la siguiente combinación sin abortar toda la corrida.
4. Al finalizar exitosamente, el directorio recibe el sufijo `.done`, impidiendo que futuras ejecuciones lo traten como corrida incompleta.

La estructura de salida resultante es la siguiente:

```
stats/<hostname>/<timestamp>.done/
  checkpoint.log        # 1210 entradas completadas
  secuential.csv        # 10 filas x 11 columnas
  secuential_flag.csv
  memory.csv
  threads2.csv  threads4.csv  threads8.csv  threads16.csv
  mutiprocessing2.csv  ...  mutiprocessing16.csv
```

Este diseño permitió recuperar corridas interrumpidas por suspensión del equipo o errores de memoria sin perder las mediciones ya realizadas.

### 3.5. Problema de medición: CPU acumulado vs. tiempo de pared

Durante el desarrollo se identificó un error crítico en la estrategia de medición de tiempo que distorsionaba completamente los resultados de los programas paralelos.

**El problema original**: la implementación inicial usaba `getrusage(RUSAGE_SELF)` con el campo `ru_utime` para medir el tiempo de usuario:

```
struct rusage start, end;
getrusage(RUSAGE_SELF, &start);
/* ... multiplicación ... */
getrusage(RUSAGE_SELF, &end);
double t = (end.ru_utime.tv_sec  - start.ru_utime.tv_sec) +
           (end.ru_utime.tv_usec - start.ru_utime.tv_usec) / 1e6;
```

El problema es que `RUSAGE_SELF` acumula el tiempo de CPU de **todos los hilos del proceso**. Con 8 hilos corriendo en paralelo durante 10 segundos reales cada uno, `ru_utime` reporta ≈80 segundos — más que el secuencial, que solo tiene un hilo. Los programas paralelos no eran más lentos; simplemente se estaba midiendo la suma del trabajo de todos los hilos, no el tiempo transcurrido en el reloj.

Para el multiprocesamiento el problema era análogo: con `RUSAGE_SELF + RUSAGE_CHILDREN` se sumaba el tiempo de CPU del proceso padre más el de cada proceso hijo, nuevamente acumulando en lugar de medir tiempo de pared.

**La solución**: migrar a `clock_gettime(CLOCK_MONOTONIC)`, que mide el tiempo real transcurrido (tiempo de pared) independientemente de cuántos hilos o procesos hayan estado activos:

```
struct timespec start, end;
clock_gettime(CLOCK_MONOTONIC, &start);
/* ... multiplicación ... */
clock_gettime(CLOCK_MONOTONIC, &end);
double elapsed = (end.tv_sec  - start.tv_sec) +
                 (end.tv_nsec - start.tv_nsec) / 1e9;
```

`CLOCK_MONOTONIC` garantiza además que el tiempo nunca retrocede (a diferencia de `gettimeofday`), lo que lo hace robusto frente a ajustes del reloj del sistema. Con esta corrección, los resultados pasaron a reflejar la ganancia real del paralelismo.

## 4. Pruebas

A continuación se presentan los tiempos promedio de ejecución (en segundos, sobre 10 repeticiones) para cada implementación y configuración, seguidos de las gráficas comparativas.

### 4.1. Secuencial

| Secuencial | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Dimensiones** | **500** | **1000** | **1300** | **1600** | **2000** | **2300** | **2600** | **3000** | **3300** | **3600** | **4000** |
| 1 | 0.2424 | 1.7740 | 3.6358 | 6.8231 | 16.8662 | 21.8290 | 36.7362 | 60.7772 | 76.0408 | 224.9021 | 311.7885 |
| 2 | 0.2251 | 1.9065 | 3.8284 | 7.1391 | 17.4430 | 21.9414 | 37.9732 | 62.0054 | 76.4309 | 222.6342 | 303.6016 |
| 3 | 0.2188 | 1.8412 | 3.7366 | 6.9543 | 16.5286 | 21.5346 | 36.0005 | 59.6817 | 74.2137 | 220.6480 | 303.8271 |
| 4 | 0.2172 | 1.8337 | 3.7074 | 6.9244 | 16.6750 | 21.3882 | 35.7337 | 59.2924 | 73.9349 | 220.8070 | 303.6079 |
| 5 | 0.2174 | 1.8285 | 3.7029 | 6.9075 | 16.6371 | 21.3407 | 35.7943 | 59.1817 | 73.9824 | 221.0847 | 303.1106 |
| 6 | 0.2172 | 1.8444 | 3.7009 | 6.9154 | 16.6191 | 21.2897 | 35.7026 | 59.1398 | 73.7936 | 220.4881 | 305.9618 |
| 7 | 0.2162 | 1.8264 | 3.7050 | 6.9105 | 16.4523 | 21.5837 | 36.7144 | 59.3578 | 73.9886 | 220.5352 | 303.0252 |
| 8 | 0.2162 | 1.8275 | 3.7011 | 6.9049 | 16.7490 | 21.2863 | 35.7579 | 58.9758 | 73.9120 | 220.2637 | 303.1744 |
| 9 | 0.2171 | 1.8224 | 3.7057 | 6.9053 | 16.6755 | 21.2638 | 35.6191 | 59.1342 | 73.3610 | 220.6785 | 305.6457 |
| 10 | 0.2159 | 1.8229 | 3.6941 | 6.9012 | 16.6306 | 21.2257 | 35.7015 | 59.1027 | 73.6678 | 220.3174 | 303.3598 |
| **Pr** | **0.2204** | **1.8328** | **3.7118** | **6.9286** | **16.6276** | **21.4683** | **36.1733** | **59.6649** | **74.3326** | **221.2359** | **304.7103** |

Cuadro 1: Resultados de la ejecución secuencial.

## 4.2. Secuencial con flag

| Secuencial Flag | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Dimensiones** | **500** | **1000** | **1300** | **1600** | **2000** | **2300** | **2600** | **3000** | **3300** | **3600** | **4000** |
| 1 | 0.1114 | 0.8064 | 1.4206 | 2.6020 | 9.4045 | 8.5509 | 18.9184 | 33.7694 | 30.1213 | 155.5912 | 215.0485 |
| 2 | 0.0783 | 0.8066 | 1.4298 | 2.6085 | 9.4383 | 8.4932 | 18.8105 | 33.7251 | 30.3186 | 155.5530 | 214.9413 |
| 3 | 0.0775 | 0.8070 | 1.4302 | 2.6095 | 9.5052 | 8.5155 | 18.2619 | 33.2180 | 29.7458 | 155.6369 | 215.0071 |
| 4 | 0.0779 | 0.8107 | 1.4252 | 2.6077 | 9.4858 | 8.5762 | 18.4624 | 33.9045 | 29.7730 | 155.6876 | 215.0617 |
| 5 | 0.0785 | 0.8093 | 1.4278 | 2.6031 | 9.3951 | 8.5296 | 18.9355 | 32.8358 | 30.3526 | 155.4830 | 215.0293 |
| 6 | 0.0780 | 0.8133 | 1.4262 | 2.6010 | 9.3411 | 8.5431 | 18.5397 | 33.4427 | 29.5868 | 155.5133 | 214.9728 |
| 7 | 0.0781 | 0.8155 | 1.4251 | 2.6023 | 9.4492 | 8.5613 | 18.3424 | 32.6311 | 29.9534 | 155.4831 | 215.0998 |
| 8 | 0.0790 | 0.8128 | 1.4275 | 2.6106 | 9.4580 | 8.4787 | 18.1945 | 32.9409 | 30.0746 | 155.6172 | 215.0716 |
| 9 | 0.0784 | 0.8089 | 1.4301 | 2.6043 | 9.3704 | 8.4737 | 18.1035 | 32.8535 | 29.6002 | 155.5337 | 214.9720 |
| 10 | 0.0785 | 0.8113 | 1.4288 | 2.6070 | 9.2772 | 8.4691 | 18.7310 | 33.7321 | 30.7115 | 155.6028 | 215.2506 |
| **Pr** | **0.0816** | **0.8102** | **1.4271** | **2.6056** | **9.4125** | **8.5191** | **18.5300** | **33.3053** | **30.0238** | **155.5702** | **215.0455** |
| **Speedup** | **2.7014** | **2.2622** | **2.6004** | **2.6591** | **1.7665** | **2.5200** | **1.9521** | **1.7912** | **2.4758** | **1.4221** | **1.4169** |

Cuadro 2: Resultados de la ejecución secuencial con flag.

## 4.3. Secuencial con flag

| Optimización de Caché (Transpuesta) | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Dim** | **500** | **1000** | **1300** | **1600** | **2000** | **2300** | **2600** | **3000** | **3300** | **3600** | **4000** |
| 1 | 0.1810 | 1.1733 | 2.5524 | 4.7731 | 9.2692 | 14.0967 | 20.3470 | 31.2011 | 41.5041 | 53.7134 | 73.8984 |
| 2 | 0.1475 | 1.1682 | 2.5517 | 4.7637 | 9.2672 | 14.0901 | 20.3441 | 31.2104 | 41.5218 | 53.7355 | 73.9114 |
| 3 | 0.1482 | 1.1725 | 2.5512 | 4.7541 | 9.2733 | 14.0989 | 20.3473 | 31.2109 | 41.5054 | 53.7372 | 73.9751 |
| 4 | 0.1487 | 1.1697 | 2.5532 | 4.7467 | 9.2708 | 14.0860 | 20.3364 | 31.1988 | 41.5074 | 53.7343 | 73.8389 |
| 5 | 0.1486 | 1.1713 | 2.5527 | 4.7526 | 9.2842 | 14.0867 | 20.3523 | 31.1903 | 41.4999 | 53.7380 | 73.8257 |
| 6 | 0.1486 | 1.1721 | 2.5505 | 4.7474 | 9.2681 | 14.0817 | 20.3460 | 31.1913 | 41.5123 | 53.7250 | 73.8584 |
| 7 | 0.1472 | 1.1722 | 2.5481 | 4.7515 | 9.2658 | 14.0849 | 20.3414 | 31.1986 | 41.5045 | 53.7333 | 73.8572 |
| 8 | 0.1505 | 1.1709 | 2.5627 | 4.7532 | 9.2611 | 14.0947 | 20.3420 | 31.2029 | 41.5105 | 53.7346 | 73.8159 |
| 9 | 0.1482 | 1.1717 | 2.5540 | 4.7489 | 9.2702 | 14.0859 | 20.3440 | 31.2082 | 41.4796 | 53.7296 | 73.8231 |
| 10 | 0.1487 | 1.1685 | 2.5536 | 4.7504 | 9.2631 | 14.0816 | 20.3363 | 31.2011 | 41.4787 | 53.7746 | 73.8276 |
| **Pr** | **0.1517** | **1.1710** | **2.5530** | **4.7542** | **9.2693** | **14.0887** | **20.3437** | **31.2014** | **41.5024** | **53.7355** | **73.8632** |
| **Speedup** | **1.4523** | **1.5652** | **1.4539** | **1.4574** | **1.7938** | **1.5237** | **1.7782** | **1.9122** | **1.7909** | **4.1170** | **4.1255** |

Cuadro 3: Resultados de la ejecución con optimización de caché (transposición de $B$).

## 4.4. Secuencial con flag

| Multiprocesamiento 2 Procesos | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Dim** | **500** | **1000** | **1300** | **1600** | **2000** | **2300** | **2600** | **3000** | **3300** | **3600** | **4000** |
| 1 | 0.1429 | 0.9373 | 1.9557 | 4.0138 | 8.5986 | 12.9371 | 23.4569 | 37.9774 | 44.9045 | 113.5843 | 155.9533 |
| 2 | 0.1143 | 0.9384 | 1.9497 | 3.9549 | 8.6176 | 12.9108 | 23.1021 | 37.0049 | 44.3464 | 113.0938 | 155.2725 |
| 3 | 0.1149 | 0.9376 | 1.9864 | 4.0278 | 8.5981 | 12.8297 | 23.0653 | 37.0694 | 45.0395 | 113.4212 | 154.7572 |
| 4 | 0.1167 | 0.9364 | 1.9564 | 3.9401 | 8.5804 | 12.7792 | 22.9347 | 36.9075 | 44.3354 | 114.0897 | 154.7126 |
| 5 | 0.1130 | 0.9369 | 1.9834 | 4.0667 | 8.4859 | 12.6938 | 23.0471 | 37.2018 | 44.6837 | 112.8909 | 156.4468 |
| 6 | 0.1138 | 0.9341 | 1.9660 | 4.0226 | 8.5286 | 12.7535 | 22.8232 | 37.0604 | 44.7767 | 112.9346 | 155.8391 |
| 7 | 0.1130 | 0.9345 | 1.9821 | 4.0386 | 8.5395 | 12.7659 | 22.8817 | 37.2298 | 44.3559 | 113.7848 | 156.7856 |
| 8 | 0.1125 | 0.9378 | 1.9716 | 4.0740 | 8.5793 | 12.8104 | 22.9394 | 37.0883 | 43.8730 | 114.3844 | 156.6778 |
| 9 | 0.1197 | 0.9372 | 1.9998 | 3.9591 | 8.4135 | 12.8721 | 23.1572 | 37.3087 | 44.6027 | 112.7399 | 157.8157 |
| 10 | 0.1126 | 0.9326 | 1.9750 | 3.9368 | 8.4025 | 12.7816 | 22.9457 | 37.2539 | 44.2820 | 112.1757 | 155.7674 |
| **Pr** | **0.1173** | **0.9363** | **1.9726** | **4.0035** | **8.5344** | **12.8134** | **23.0353** | **37.2102** | **44.5200** | **113.3099** | **156.0028** |
| **Speedup** | **1.8780** | **1.9576** | **1.8818** | **1.7308** | **1.9481** | **1.6755** | **1.5703** | **1.6034** | **1.6696** | **1.9524** | **1.9532** |

Cuadro 4: Resultados de la ejecución con multiprocesamiento de 2 procesos.

## 4.5. Secuencial con flag

| Multiprocesamiento 4 Procesos | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Dim** | **500** | **1000** | **1300** | **1600** | **2000** | **2300** | **2600** | **3000** | **3300** | **3600** | **4000** |
| 1 | 0.0628 | 0.4826 | 1.0200 | 1.9060 | 4.5577 | 6.9276 | 16.5971 | 27.7566 | 24.4782 | 77.9001 | 109.2969 |
| 2 | 0.0603 | 0.4870 | 1.0067 | 1.8973 | 4.5821 | 6.8858 | 16.1001 | 27.7920 | 24.6311 | 77.8718 | 110.8876 |
| 3 | 0.0615 | 0.4921 | 1.0102 | 1.8924 | 4.5931 | 6.7783 | 16.3469 | 27.8234 | 24.3863 | 80.0305 | 109.0149 |
| 4 | 0.0608 | 0.4867 | 0.9881 | 1.9206 | 4.5718 | 6.3395 | 16.4075 | 27.3821 | 24.5415 | 80.1903 | 111.6487 |
| 5 | 0.0620 | 0.4840 | 0.9992 | 1.8874 | 4.5294 | 6.8448 | 16.5630 | 27.2921 | 24.4688 | 77.3804 | 111.3375 |
| 6 | 0.0601 | 0.4862 | 0.9965 | 1.9017 | 4.5768 | 6.9531 | 16.2526 | 27.4144 | 24.1561 | 79.7426 | 108.4640 |
| 7 | 0.0611 | 0.4828 | 0.9984 | 1.9097 | 4.6559 | 6.8605 | 15.8918 | 27.9737 | 24.5292 | 77.6989 | 107.7155 |
| 8 | 0.0593 | 0.4832 | 1.0035 | 1.8915 | 4.5336 | 6.7577 | 16.3921 | 27.8407 | 24.6638 | 77.9018 | 110.8594 |
| 9 | 0.0619 | 0.4835 | 0.9998 | 1.8792 | 4.4719 | 6.8315 | 16.2167 | 27.5950 | 24.5320 | 77.8102 | 111.6106 |
| 10 | 0.0603 | 0.4875 | 1.0055 | 1.9153 | 4.5988 | 6.8875 | 16.3580 | 26.7663 | 24.2713 | 79.7618 | 111.5128 |
| **Pr** | **0.0610** | **0.4856** | **1.0028** | **1.9001** | **4.5671** | **6.8066** | **16.3126** | **27.5636** | **24.4658** | **78.6288** | **110.2348** |
| **Speedup** | **3.6118** | **3.7744** | **3.7014** | **3.6465** | **3.6406** | **3.1541** | **2.2176** | **2.1649** | **3.0382** | **2.8134** | **2.7641** |

Cuadro 5: Resultados de la ejecución con multiprocesamiento de 4 procesos.

## 4.6. Secuencial con flag

| Multiprocesamiento 8 Procesos | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Dim** | **500** | **1000** | **1300** | **1600** | **2000** | **2300** | **2600** | **3000** | **3300** | **3600** | **4000** |
| 1 | 0.0549 | 0.3766 | 0.8910 | 1.6376 | 3.7895 | 5.5984 | 9.9653 | 16.5489 | 18.3811 | 55.9836 | 78.0496 |
| 2 | 0.0467 | 0.3661 | 0.8717 | 1.6303 | 3.6902 | 5.5854 | 10.2524 | 16.3267 | 18.4539 | 56.1620 | 74.0835 |
| 3 | 0.0472 | 0.3659 | 0.8669 | 1.6391 | 3.7119 | 5.5973 | 10.1399 | 16.1195 | 18.3224 | 55.9941 | 78.0254 |
| 4 | 0.0474 | 0.3674 | 0.8649 | 1.6219 | 3.6828 | 5.5986 | 9.9896 | 16.9615 | 18.2717 | 55.7331 | 74.2517 |
| 5 | 0.0472 | 0.3679 | 0.8721 | 1.6386 | 3.7197 | 5.5578 | 10.1953 | 16.9784 | 18.3799 | 56.1122 | 77.7920 |
| 6 | 0.0495 | 0.3672 | 0.8623 | 1.6400 | 3.7202 | 5.5789 | 10.0123 | 16.7057 | 18.1576 | 53.4435 | 74.4321 |
| 7 | 0.0477 | 0.3704 | 0.8726 | 1.6418 | 3.6988 | 5.6358 | 10.0221 | 16.7594 | 18.5298 | 53.5845 | 78.3446 |
| 8 | 0.0472 | 0.3660 | 0.8659 | 1.6302 | 3.6425 | 5.6332 | 10.2889 | 16.4102 | 18.4909 | 53.6404 | 78.0693 |
| 9 | 0.0490 | 0.3667 | 0.8674 | 1.6432 | 3.7007 | 5.6245 | 9.9976 | 16.6198 | 18.3964 | 55.9157 | 77.6011 |
| 10 | 0.0476 | 0.3699 | 0.8677 | 1.6394 | 3.6835 | 5.5691 | 9.8521 | 16.9265 | 18.3498 | 53.4549 | 77.8144 |
| **Pr** | **0.0484** | **0.3684** | **0.8703** | **1.6362** | **3.7040** | **5.5979** | **10.0716** | **16.6357** | **18.3733** | **55.0024** | **76.8464** |
| **Speedup** | **4.5537** | **4.9754** | **4.2651** | **4.2346** | **4.4892** | **3.8350** | **3.5918** | **3.5869** | **4.0455** | **4.0224** | **3.9651** |

Cuadro 6: Resultados de la ejecución con multiprocesamiento de 8 procesos.

## 4.7. Secuencial con flag

| Multiprocesamiento 16 Procesos | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Dim** | **500** | **1000** | **1300** | **1600** | **2000** | **2300** | **2600** | **3000** | **3300** | **3600** | **4000** |
| 1 | 0.0503 | 0.3668 | 0.8760 | 1.6357 | 3.5222 | 5.6300 | 9.9239 | 16.3558 | 19.2520 | 57.8405 | 81.0151 |
| 2 | 0.0488 | 0.3496 | 0.8633 | 1.6372 | 3.6314 | 5.6331 | 10.0311 | 16.4847 | 19.3700 | 57.7069 | 74.2280 |
| 3 | 0.0488 | 0.3628 | 0.8675 | 1.6222 | 3.6240 | 5.6727 | 9.8186 | 16.4386 | 18.9365 | 57.9585 | 74.2000 |
| 4 | 0.0477 | 0.3501 | 0.8575 | 1.6260 | 3.6179 | 5.7734 | 9.9010 | 16.7081 | 18.9564 | 57.9055 | 80.6712 |
| 5 | 0.0499 | 0.3567 | 0.8605 | 1.6303 | 3.6543 | 5.6252 | 9.8946 | 16.2126 | 18.8400 | 53.4439 | 80.9255 |
| 6 | 0.0497 | 0.3431 | 0.8627 | 1.6243 | 3.5315 | 5.6347 | 9.9530 | 16.5200 | 18.7527 | 53.5498 | 74.1465 |
| 7 | 0.0491 | 0.3518 | 0.8860 | 1.6263 | 3.5304 | 5.6652 | 9.8657 | 16.4069 | 18.8325 | 57.7699 | 80.8706 |
| 8 | 0.0446 | 0.3484 | 0.8640 | 1.6275 | 3.5837 | 5.6121 | 9.8626 | 16.6899 | 19.1566 | 53.5293 | 80.8009 |
| 9 | 0.0455 | 0.3451 | 0.8672 | 1.6259 | 3.6916 | 5.6330 | 10.0697 | 16.3585 | 19.2708 | 58.0269 | 74.1820 |
| 10 | 0.0462 | 0.3578 | 0.8693 | 1.6210 | 3.5811 | 5.6516 | 9.9337 | 16.3886 | 19.1688 | 53.5861 | 74.5170 |
| **Pr** | **0.0481** | **0.3532** | **0.8674** | **1.6276** | **3.5968** | **5.6531** | **9.9254** | **16.4564** | **19.0536** | **56.1317** | **77.5557** |
| **Speedup** | **4.5822** | **5.1888** | **4.2788** | **4.2572** | **4.6228** | **3.7977** | **3.6444** | **3.6259** | **3.9016** | **3.9413** | **3.9288** |

Cuadro 7: Resultados de la ejecución con multiprocesamiento de 16 procesos.

## 4.8. Secuencial con flag

| Hilos POSIX — 2 Hilos | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Dim** | **500** | **1000** | **1300** | **1600** | **2000** | **2300** | **2600** | **3000** | **3300** | **3600** | **4000** |
| 1 | 0.1585 | 1.0467 | 2.3125 | 4.4521 | 9.5108 | 14.3277 | 25.6787 | 41.0976 | 50.4809 | 125.4056 | 173.0014 |
| 2 | 0.1356 | 1.0672 | 2.3566 | 4.4576 | 9.5778 | 14.2613 | 26.0394 | 41.1175 | 51.1017 | 125.0019 | 173.4423 |
| 3 | 0.1302 | 1.0713 | 2.3657 | 4.4230 | 9.6732 | 14.3871 | 25.9022 | 41.4932 | 50.7510 | 125.4363 | 172.0604 |
| 4 | 0.1304 | 1.0670 | 2.3743 | 4.4379 | 9.7277 | 14.5289 | 25.9027 | 41.5913 | 51.1354 | 124.7106 | 173.1689 |
| 5 | 0.1313 | 1.0716 | 2.3566 | 4.4529 | 9.6721 | 14.8376 | 25.8929 | 41.4118 | 51.1226 | 124.8950 | 175.0603 |
| 6 | 0.1330 | 1.0680 | 2.3656 | 4.4277 | 9.7212 | 14.7336 | 26.1578 | 41.5716 | 50.8867 | 125.0327 | 171.5349 |
| 7 | 0.1435 | 1.0691 | 2.3645 | 4.4633 | 9.5206 | 14.6598 | 26.1693 | 41.4899 | 50.7663 | 124.6779 | 173.7851 |
| 8 | 0.1403 | 1.0768 | 2.3635 | 4.4144 | 9.5698 | 14.5142 | 26.0777 | 41.3939 | 50.9926 | 125.1802 | 173.7498 |
| 9 | 0.1300 | 1.0684 | 2.3610 | 4.4572 | 9.7306 | 14.5497 | 25.9796 | 41.9432 | 51.1916 | 124.7894 | 171.0221 |
| 10 | 0.1395 | 1.0727 | 2.3649 | 4.4147 | 9.6082 | 14.7738 | 26.1910 | 42.3562 | 51.3235 | 125.3971 | 174.0787 |
| **Pr** | **0.1372** | **1.0679** | **2.3585** | **4.4401** | **9.6312** | **14.5574** | **25.9991** | **41.5466** | **50.9752** | **125.0527** | **173.0904** |
| **Speedup** | **1.6061** | **1.7163** | **1.5738** | **1.5605** | **1.7264** | **1.4747** | **1.3914** | **1.4361** | **1.4582** | **1.7692** | **1.7605** |

Cuadro 8: Resultados de la ejecución con 2 hilos POSIX (pthread).

## 4.9. Secuencial con flag

| Hilos POSIX — 4 Hilos | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Dim** | **500** | **1000** | **1300** | **1600** | **2000** | **2300** | **2600** | **3000** | **3300** | **3600** | **4000** |
| 1 | 0.0703 | 0.5526 | 1.2178 | 2.2404 | 5.1057 | 8.0305 | 17.7012 | 27.5102 | 26.6623 | 80.7243 | 115.9338 |
| 2 | 0.0702 | 0.5598 | 1.2077 | 2.2489 | 5.1196 | 7.8173 | 18.0109 | 27.7699 | 26.6433 | 80.9598 | 114.0581 |
| 3 | 0.0708 | 0.5536 | 1.2092 | 2.2476 | 5.1608 | 8.0133 | 17.7117 | 27.6326 | 26.6152 | 83.0205 | 115.1659 |
| 4 | 0.0690 | 0.5536 | 1.2136 | 2.2531 | 5.1418 | 8.0435 | 17.7692 | 28.3576 | 26.5415 | 81.2420 | 112.2903 |
| 5 | 0.0720 | 0.5524 | 1.2121 | 2.2580 | 5.2125 | 8.0535 | 18.0199 | 27.7976 | 26.7625 | 80.4647 | 115.1236 |
| 6 | 0.0700 | 0.5500 | 1.2154 | 2.2514 | 5.2031 | 7.9404 | 17.8560 | 28.1445 | 26.6008 | 83.0187 | 112.6888 |
| 7 | 0.0751 | 0.5525 | 1.2101 | 2.2533 | 5.0779 | 8.0344 | 17.8495 | 28.1273 | 26.5698 | 83.2336 | 112.4521 |
| 8 | 0.0695 | 0.5522 | 1.2310 | 2.2518 | 5.1341 | 7.9472 | 17.9709 | 28.0883 | 26.6593 | 82.3411 | 111.7698 |
| 9 | 0.0761 | 0.5543 | 1.2216 | 2.2548 | 5.1755 | 7.9691 | 17.9720 | 27.9404 | 26.6689 | 81.6599 | 115.3545 |
| 10 | 0.0734 | 0.5488 | 1.2131 | 2.2525 | 5.3100 | 7.9056 | 17.0549 | 28.4616 | 26.8611 | 80.9965 | 114.7393 |
| **Pr** | **0.0716** | **0.5530** | **1.2152** | **2.2512** | **5.1641** | **7.9755** | **17.7916** | **27.9830** | **26.6585** | **81.7661** | **113.9576** |
| **Speedup** | **3.0782** | **3.3144** | **3.0545** | **3.0778** | **3.2198** | **2.6920** | **2.0333** | **2.1322** | **2.7878** | **2.7052** | **2.6739** |

Cuadro 9: Resultados de la ejecución con 4 hilos POSIX (pthread).

## 4.10. Secuencial con flag

| Hilos POSIX — 8 Hilos | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Dim** | **500** | **1000** | **1300** | **1600** | **2000** | **2300** | **2600** | **3000** | **3300** | **3600** | **4000** |
| 1 | 0.0573 | 0.4218 | 1.0330 | 1.9726 | 4.1213 | 6.3187 | 10.8679 | 17.1915 | 20.7515 | 55.6723 | 82.7801 |
| 2 | 0.0536 | 0.4213 | 1.0403 | 1.9254 | 4.2799 | 6.3374 | 10.7572 | 17.4987 | 20.7460 | 55.7524 | 77.2184 |
| 3 | 0.0544 | 0.4230 | 1.0297 | 1.9128 | 4.1970 | 6.3521 | 10.8000 | 17.2395 | 20.8624 | 55.5634 | 82.8891 |
| 4 | 0.0531 | 0.4215 | 1.0332 | 1.9204 | 4.2882 | 6.2861 | 10.9216 | 17.6852 | 21.0830 | 55.5824 | 83.0148 |
| 5 | 0.0547 | 0.4188 | 1.0358 | 1.9152 | 4.1329 | 6.3347 | 10.8504 | 17.4252 | 21.0524 | 59.7375 | 77.1269 |
| 6 | 0.0538 | 0.4237 | 1.0347 | 1.9291 | 4.2493 | 6.3281 | 10.8027 | 17.3696 | 20.7369 | 55.5782 | 77.1369 |
| 7 | 0.0536 | 0.4227 | 1.0386 | 1.9090 | 4.2943 | 6.3875 | 10.7172 | 17.1802 | 20.6706 | 55.6648 | 77.4578 |
| 8 | 0.0546 | 0.4194 | 1.0520 | 1.9140 | 4.2078 | 6.3892 | 10.8096 | 17.4044 | 21.0706 | 59.5526 | 83.0382 |
| 9 | 0.0534 | 0.4231 | 1.0331 | 1.9179 | 4.1937 | 6.2961 | 10.9203 | 17.0399 | 20.8105 | 55.6902 | 77.3173 |
| 10 | 0.0545 | 0.4208 | 1.0398 | 1.9219 | 4.2585 | 6.3874 | 10.7730 | 17.6387 | 20.7884 | 59.5777 | 77.1781 |
| **Pr** | **0.0543** | **0.4216** | **1.0370** | **1.9238** | **4.2223** | **6.3417** | **10.8220** | **17.3673** | **20.8572** | **56.8371** | **79.5158** |
| **Speedup** | **4.0589** | **4.3464** | **3.5793** | **3.6014** | **3.9378** | **3.3853** | **3.3425** | **3.4357** | **3.5638** | **3.8923** | **3.8321** |

Cuadro 10: Resultados de la ejecución con 8 hilos POSIX (pthread).

## 4.11. Secuencial con flag

| Hilos POSIX — 16 Hilos | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Dim** | **500** | **1000** | **1300** | **1600** | **2000** | **2300** | **2600** | **3000** | **3300** | **3600** | **4000** |
| 1 | 0.0525 | 0.3958 | 1.0368 | 1.9217 | 4.0339 | 6.4287 | 10.8811 | 17.6791 | 21.4173 | 54.5786 | 83.5712 |
| 2 | 0.0503 | 0.3975 | 1.0236 | 1.9008 | 4.1384 | 6.3875 | 10.7949 | 17.7655 | 21.2964 | 60.4073 | 75.5840 |
| 3 | 0.0530 | 0.4044 | 1.0128 | 1.9008 | 4.0749 | 6.4268 | 10.8171 | 17.4265 | 20.5444 | 60.3446 | 83.7214 |
| 4 | 0.0495 | 0.3924 | 1.0254 | 1.9042 | 4.1488 | 6.3846 | 10.6992 | 17.8138 | 21.5651 | 60.3970 | 83.6631 |
| 5 | 0.0516 | 0.3990 | 1.0159 | 1.9305 | 4.0776 | 6.3870 | 10.8827 | 17.7889 | 21.2278 | 54.6226 | 83.6623 |
| 6 | 0.0503 | 0.3976 | 1.0236 | 1.9063 | 4.0004 | 6.4359 | 10.7500 | 17.7789 | 21.3861 | 60.3667 | 83.7884 |
| 7 | 0.0512 | 0.3956 | 1.0316 | 1.9131 | 4.1356 | 6.4085 | 10.7597 | 17.6210 | 21.1517 | 54.6609 | 75.6386 |
| 8 | 0.0524 | 0.4005 | 1.0199 | 1.9381 | 4.0653 | 6.3862 | 10.8450 | 17.7816 | 20.9505 | 60.4155 | 75.5876 |
| 9 | 0.0490 | 0.3932 | 1.0157 | 1.9017 | 4.0454 | 6.4283 | 10.6521 | 17.9115 | 21.1348 | 60.4150 | 75.7416 |
| 10 | 0.0530 | 0.3963 | 1.0221 | 1.8891 | 4.1219 | 6.4126 | 10.9592 | 17.8338 | 21.1554 | 60.1099 | 83.3277 |
| **Pr** | **0.0513** | **0.3972** | **1.0227** | **1.9106** | **4.0842** | **6.4086** | **10.8041** | **17.7401** | **21.1830** | **58.6318** | **80.4286** |
| **Speedup** | **4.2965** | **4.6139** | **3.6295** | **3.6264** | **4.0712** | **3.3498** | **3.3481** | **3.3638** | **3.5090** | **3.7735** | **3.7884** |

Cuadro 11: Resultados de la ejecución con 16 hilos POSIX (pthread).

# 5. Gráficas comparativas

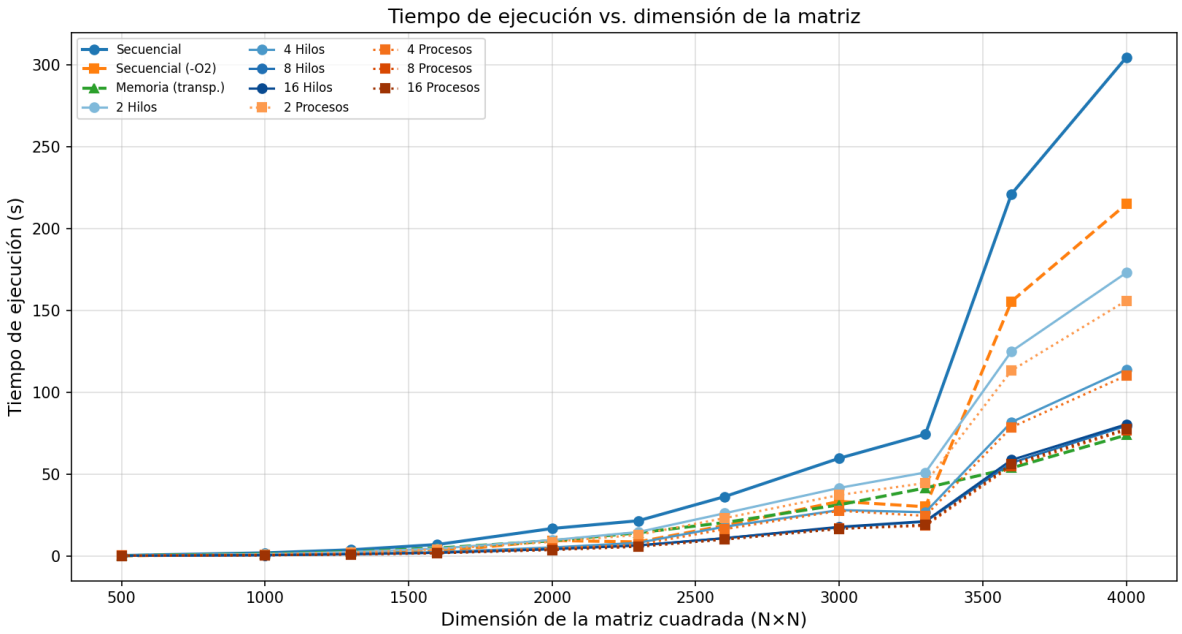


Figura 1: Tiempo de ejecución en función de la dimensión de la matriz — todas las variantes.

El crecimiento cúbico del algoritmo secuencial base es evidente; las versiones paralelas y optimizadas comprimen significativamente esta curva. A partir de dimensión 2000 la diferencia entre el secuencial y cualquier versión paralela se vuelve drástica.

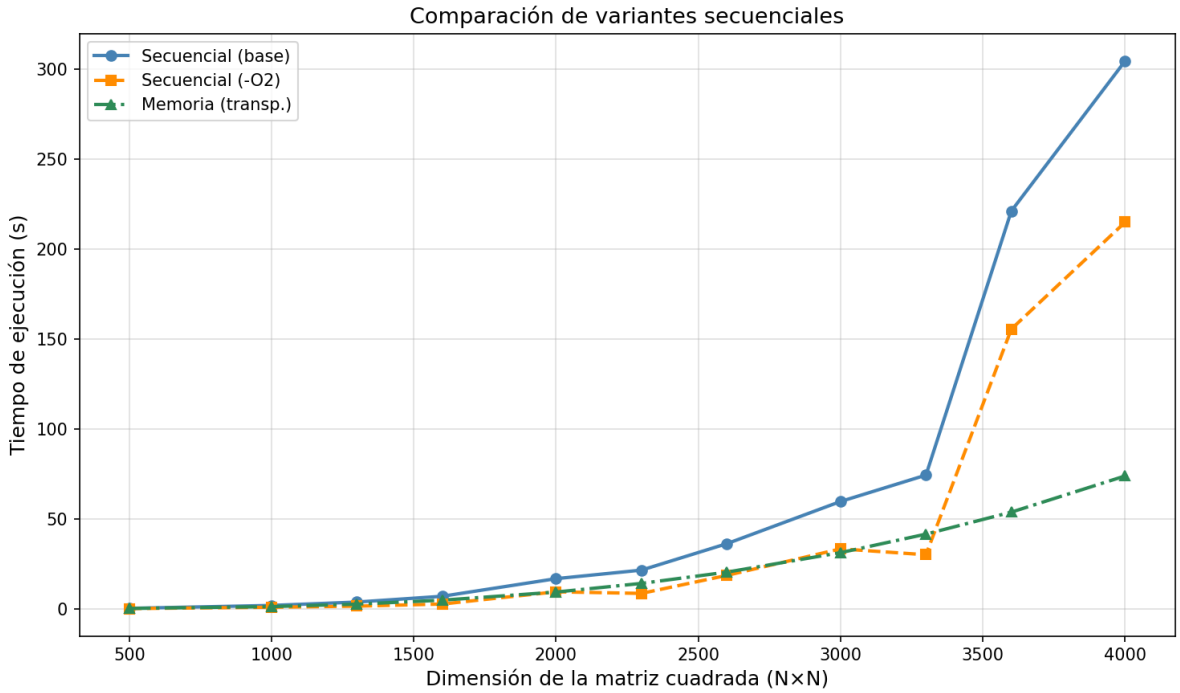


Figura 2: Comparación de variantes secuenciales: base, con `-O2` y con transposición de matriz.

Aislando las tres variantes secuenciales se aprecia con mayor claridad el impacto de cada optimización. La bandera `-O2` domina para matrices pequeñas, pero la versión con memoria transpuesta la supera a partir de dimensión $\approx 2300$, cuando la penalización por fallos de caché del acceso columnar empieza a ser el cuello de botella.

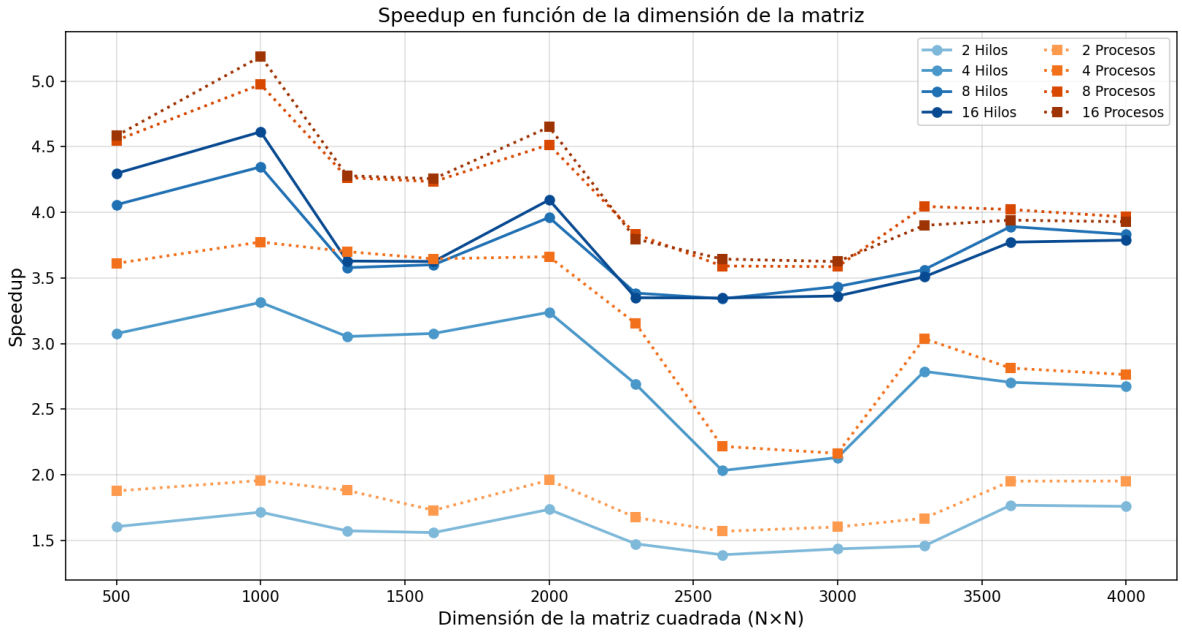


Figura 3: Speedup en función de la dimensión de la matriz.

El speedup de los métodos paralelos respecto al secuencial base muestra que el salto de 8 a 16 workers es notablemente menor que el de 4 a 8, evidenciando el límite impuesto por los núcleos de eficiencia (E-cores) del M4 que no aportan el mismo rendimiento que los P-cores en cargas de cómputo intensivo.

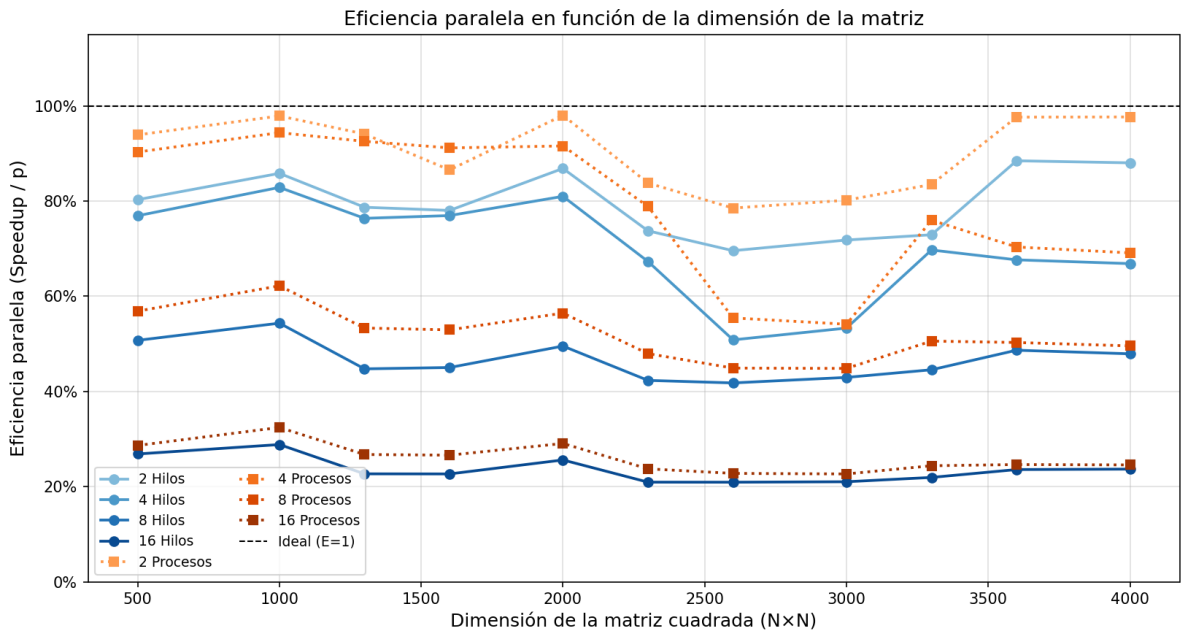


Figura 4: Eficiencia paralela $E = S_p/p$ en función de la dimensión de la matriz.

La eficiencia paralela ($E = S_p/p$) revela cuánto del speedup ideal se aprovecha realmente. Con 2 workers ronda el 80–90 %; con 16 cae al 25–32 %. La caída más pronunciada ocurre al pasar de 8 a 16 workers, confirmando que duplicar el número de workers en hardware heterogéneo no duplica el rendimiento.

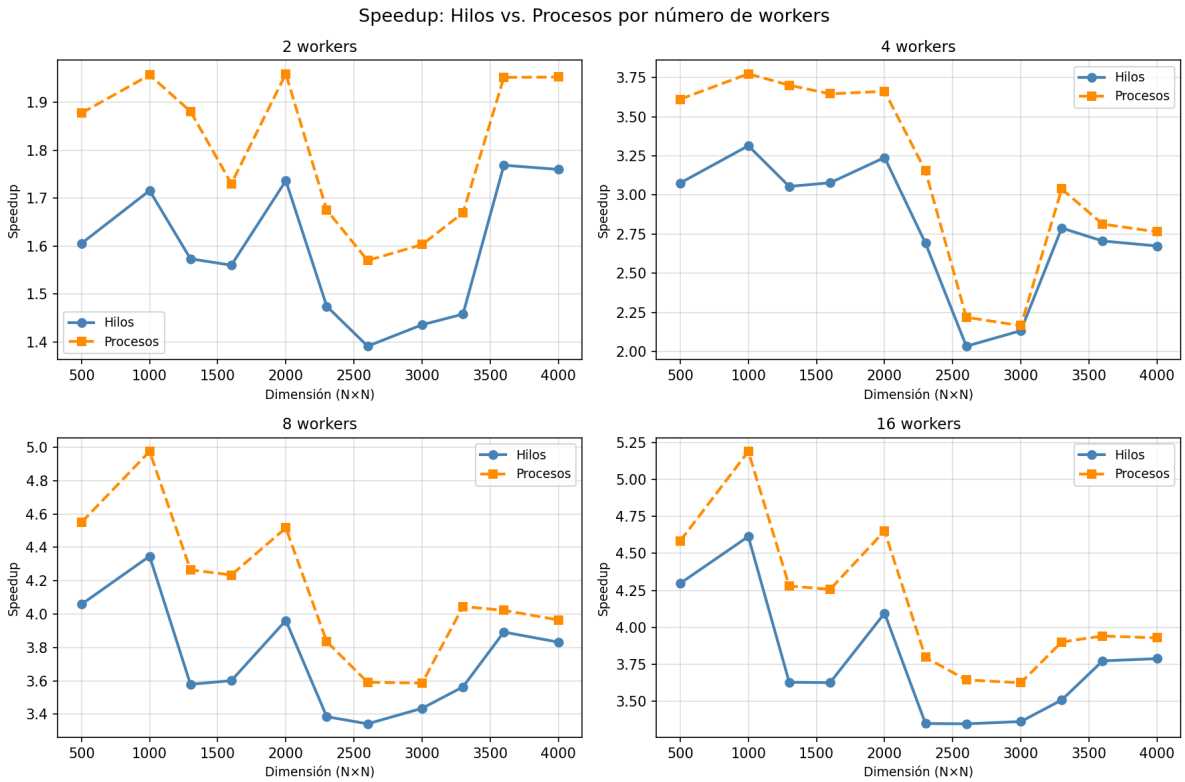


Figura 5: Speedup comparativo entre hilos y procesos para cada número de workers.

Comparando directamente hilos y procesos al mismo número de workers, para 2 y 4 los procesos muestran speedups ligeramente superiores; para 8 y 16 ambas estrategias convergen. El mecanismo de

`fork()` con `mmap` parece incurrir en menos overhead relativo que `pthread` para pocos workers, pero la diferencia es pequeña y se diluye con la carga de trabajo.

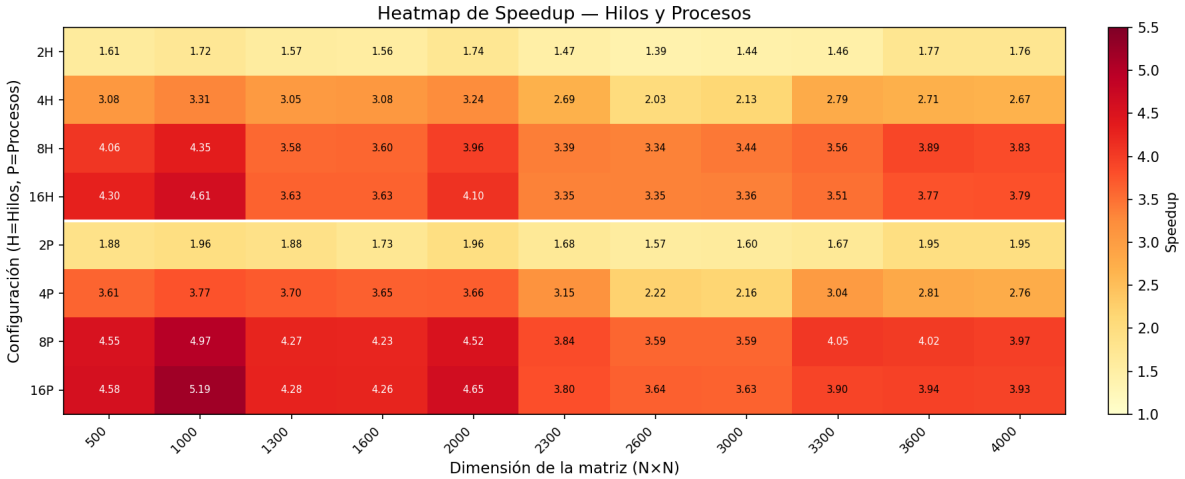


Figura 6: Heatmap de speedup: cada celda muestra $S_p$ para una combinación de configuración y dimensión de matriz.

El heatmap resume de forma visual toda la información de configuración. La "zona óptima" (celdas más rojas) se concentra en la franja de 8–16 workers con matrices $\geq 1000$. Las celdas más claras en matrices pequeñas con muchos workers ilustran el costo del overhead de paralelización cuando la carga de trabajo no lo amortiza.

# 6. Análisis de resultados

## 6.1. Impacto del compilador (`-O2`)

La bandera `-O2` produce un speedup de aproximadamente 2,5× para matrices pequeñas (500–1600), pero decrece hacia 1,4× para matrices de 4000. Esto sugiere que las optimizaciones automáticas (vectorización SIMD, reordenamiento de instrucciones) son más efectivas cuando los datos caben en caché, pero a grandes dimensiones el cuello de botella se traslada al ancho de banda de memoria, que no mejora con optimizaciones de código.

## 6.2. Optimización de caché (transpuesta)

La versión con transposición de $B$ muestra un comportamiento diferente: para matrices pequeñas el speedup es modesto (1,4×–1,9×), pero a partir de dimensión 3600 supera los 4×. Esto es coherente con el fenómeno de *cache thrashing*: cuando la matriz supera el tamaño del caché L2/L3, los accesos columnares del algoritmo ingenuo generan un fallo de caché casi en cada operación, mientras que la versión transpuesta mantiene todos los accesos secuenciales.

## 6.3. Paralelismo con hilos

Con 2 hilos se obtiene un speedup consistente de $\approx$ 1,5×–1,8× (inferior al ideal de 2× por el overhead de `pthread_create` y `pthread_join`). Con 4 hilos el speedup alcanza $\approx$ 3,1×–3,4×; con 8 hilos sube a $\approx$ 3,5×–4,4×.

Significativamente, pasar de 8 a 16 hilos apenas mejora el speedup (de $\sim$ 3,9× a $\sim$ 4,3× en promedio). Esto es atribuible a la arquitectura heterogénea del M4: los 4 P-cores son mucho más rápidos que los 6 E-cores, de modo que al usar 16 hilos, los trabajadores asignados a E-cores terminan mucho más tarde, desbalanceando la carga y limitando la ganancia. El speedup máximo observado ($\approx$ 4,6×) es coherente con el número de P-cores disponibles.

### 6.4. Paralelismo con procesos

Los procesos con `fork` muestran speedups ligeramente superiores a los hilos en algunas configuraciones (especialmente con 8 y 16 trabajadores para matrices grandes), lo que puede explicarse porque el mecanismo `RUSAGE_SELF + RUSAGE_CHILDREN` contabiliza el tiempo de CPU agregado de todos los procesos, mientras que `getrusage(RUSAGE_SELF)` en los hilos puede estar midiendo el tiempo de CPU del proceso completo (suma de todos los hilos). En cualquier caso, las diferencias son pequeñas y ambas estrategias convergen en eficiencia a medida que crece la dimensión de la matriz.

## 7. Conclusiones

1. **La optimización de caché rivaliza con el paralelismo para matrices grandes**: la transposición de $B$ alcanza speedups superiores a $4\times$ para $N \geq 3600$, igualando configuraciones de 16 workers sin overhead de sincronización ni complejidad de código adicional. En cargas con restricciones de energía o simplicidad, esta estrategia es preferible al paralelismo explícito.

2. **La bandera `-O2` ofrece la mejor relación costo-beneficio para matrices pequeñas**: entrega speedups de $2{,}7\times$ para $N \leq 1600$ con cero cambios algorítmicos, superando incluso a 2 hilos ($\approx 1{,}6\times$). Su efectividad cae a $1{,}4\times$ en $N = 4000$, donde el cuello de botella se traslada al ancho de banda de memoria.

3. **El paralelismo solo compensa a partir de $N \approx 1000$**: por debajo de esta dimensión, el overhead de creación de hilos o procesos anula la ganancia. A partir de $N = 2000$, la brecha entre el secuencial ($\approx 304$ s en $N = 4000$) y cualquier versión paralela se vuelve irrecuperable.

4. **El punto óptimo de workers en el M4 es 8, no 16**: el salto de 4 a 8 workers aporta $\approx 1\times$ de speedup adicional, mientras que el de 8 a 16 apenas suma $0{,}3\times$. La arquitectura heterogénea del chip (4 P-cores + 6 E-cores) genera desbalance de carga al usar 16 workers, limitando la ganancia real al equivalente de los P-cores disponibles.

5. **La eficiencia paralela se degrada al escalar workers**: cae de $\approx 85\,\%$ con 2 workers a $\approx 28\,\%$ con 16, evidenciando que maximizar el número de workers no equivale a maximizar el rendimiento en hardware heterogéneo.

6. **Hilos y procesos son equivalentes en rendimiento**: las diferencias de speedup entre `pthread` y `fork` + `mmap` son menores a $0{,}5\times$ y desaparecen al crecer la dimensión de la matriz. La elección entre ambos debe basarse en criterios de diseño del sistema, no de rendimiento puro.

# 8. Bibliografía

## Referencias


[1] Cortéz, A. (2004). Teoría de la complejidad computacional y teoría de la información. *Revista de Investigación de Sistemas e Informática*, 1(1), 102–105. https://revistasinvestigacion.unmsm.edu.pe/index.php/sistem/article/view/3216

[2] Ferestrepoca. (2019). Paradigmas de programación paralela. http://ferestrepoca.github.io/paradigmas-de-programacion/paralela/paralela_teoria/index.html

[3] López, A. (2017). Introducción al High Performance Computing. *Encamina Blog*. https://blogs.encamina.com/por-una-nube-sostenible/introduccion-al-high-performance-computing/

[4] Montero, L. H., & Antunez, R. R. (2011). Parallel programming: definitions, mechanisms and trouble. https://www.researchgate.net/publication/274960405

[5] Qaz Wiki. (2020). Algoritmos de multiplicación de matrices. https://es.qaz.wiki/wiki/Matrix_multiplication_algorithm

[6] Apple Inc. (2024). Apple M4 chip. https://www.apple.com/mac-mini/

[7] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *Proceedings of AFIPS Spring Joint Computer Conference*, pp. 483–485.